

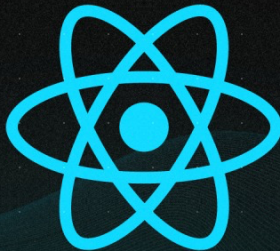


New hook: 'useOptimistic'

❤️ 1,36,98²/₃ likes

(some day)

Follow me for
other new react
features



REACT 19

coming soon...



Purpose

The useOptimistic hook enhances user experience by enabling immediate UI updates based on anticipated results, even before the server confirms the action.

This creates the illusion of a faster and more responsive application.





Swipe for code

How it works

- 1 User Initiates Action: A **user** interaction **triggers an action**, such as submitting a form or clicking a button.
- 2 Optimistic State Creation: The `useOptimistic` hook takes the **current state as input** and returns an object containing two key parts:
 - `optimisticValue`: This represents the **anticipated state** after the action is successful.
 - `updateOptimistic`: This function allows you to **update** the `optimisticValue` based on user input or other factors.
- 3 UI Update with Optimistic State: The UI component utilizes the `optimisticValue` to **reflect the expected change** on the screen.



Swipe →



- 4 Background Operation Runs: In the background, the **actual operation** (e.g., sending data to the server) is carried out.
- 5 Outcome Handling:
 - **Success**: Upon successful completion, the **optimistic state becomes the permanent state**, and the UI remains consistent.
 - **Failure**: If the operation fails, the **UI is reverted to its original state**, potentially displaying an **error message**.

swipe for code →



Swipe →



```
const { optimisticValue, updateOptimistic } = useOptimistic(count);

const handleClick = () => {
  updateOptimistic(count + 1);

  fetch('/update-count', { method: 'POST', body: JSON.stringify({
    count: count + 1 }) })
    .then(() => console.log('Count updated successfully'))
    .catch(() => console.error('Failed to update count'));
};

return (
  <div>
    <p>Count: {optimisticValue}</p>
    <button onClick={handleClick}>Increment</button>
  </div>
);
```

Diagram illustrating the sequence of events in the code:

1. Initial state (UI updated)
2. User clicks the button (UI updated)
3. Data is sent to the server (send data to server)
4. Server response is received (send data to server)
5. UI is updated with the new count (UI updated)





Real life Applications

E-commerce Shopping Carts: Update cart totals immediately after adding or removing items, providing a smooth shopping experience.



Messaging Applications: Display messages as "sent" right away, even before server confirmation, enhancing the feeling of real-time chat interaction.



Social Media Likes: Show a like animation and update the like count instantly upon clicking the "Like" button, creating a more engaging user experience.

❤️ 1,36,982 likes

(some day)



Swipe →



Subscribe on **You Tube** to
learn more



Code with Sloba

@CodewithSloba • 21.8K subscribers

Hi, my name is Slobodan (Sloba) Gajic, and I'm a JavaScript software developer. Building a

**Subscribed** 

